

8. Vue项目实战（二）基础框架开发

课程资料



 plugin.js



byelide.zip

1.68MB



源码地址：<https://github.com/encode-studio-fe/encode-byelide>



byelide-0910.zip

5.66MB



课程源码，重点看这几个分支：

- features/layer-engine
- features/material-layout

课程目标



- 初级：
 - 了解插件化机制的实现
 - 掌握表单检验与表单状态管理
- 中级：
 - 掌握插件化设计思想
 - 掌握动态表单设计思想及表单校验规则
 - 掌握物料配置核心实现思路
- 高级：
 - 掌握前端企业及项目架构设计思想
 - 掌握相关需求技术选型与方案评估
 - 掌握组件设计与封装

注意：因为是实战课，除了看懂项目架构与基础开发以外，还要着重培养项目相关描述，在面试过程中这点非常重要！

本节课我们主要从业务层面出发，让大家深度参与一个项目从需求阶段到落地的整个实施过程。



特别是之前很多同学没有参与过大型项目的，主要写后台相关 CRUD 的同学，一定一定要完全掌握本次实战课程的内容！这会直接决定你的编程思维与薪资涨幅！

课程大纲

1. 插件化机制实现物料灵活组装
2. 配置面板开发
3. 表单生成器 Form Builder 动态生成表单实现校验与提交



加载失败，请重试

课程内容

```
1      {
2        path: '/*:pathMatch(.*)*',
3        name: '404',
4        component: () => h('div', '404')
5      }
```

插件化机制实现物料灵活组装

物料的加载我们选用插件化机制实现，插件化机制在前端应用或者框架中的应用非常广泛。不论是业务开发还是框架开发，我们都能见到插件化设计。比如 webpack 基于 tappable 实现的插件化。

<https://github.com/yiminghe/async-validator/tree/master>

插件机制引入

我们先通过一个小例子来看看插件化的简单实现：

```
1  class Calculator {
2    construct(initial) {
3      this.num = initial;
4    }
5    add(num) {
6      this.num = this.num + num;
7      return this;
8    }
9    subtract(num) {
10     this.num = this.num - num;
11     return this;
12   }
13   result() {
14     return this.num;
15   }
16 }
```

```
17
18   const myCalculator = new Calculator(5);
19   myCalculator.add(5).subtract(2).result(); // 8
```

以上这个例子，我们定义了一个计算器，计算器有加减功能，但是如果我们希望增加乘除功能呢？没办法只能在类中修改代码，这是非常不灵活的，那我们可以怎么做？



做最重要的事！

计算器底座是最重要的事

计算功能作为插件，只需要注册到插件系统即可

插件化机制最重要的事是什么？没错就是插件底座，我们要开发计算器，我们只关心计算器整个底座，至于计算器有什么计算功能，交给外部，外部灵活拓展。顺着这个思路，那么计算器的各种计算方法就是插件，我们需要某个计算功能的时候只需要往插件系统中注册对应插件即可。

```
1  // 程序主体，定义程序核心逻辑是增加计算器运算能力
2  class Calculator {
3      plugins = [];
4      constructor(initial) {
5          this.num = initial;
6      }
7
8      use(plugin) {
9          this.plugins.push(plugin);
10         this[plugin.name] = plugin.calculate.bind(this);
11     }
12
13     result() {
14         return this.num;
15     }
16 }
17 // 插件声明
18 interface Plugin {
19     name: string;
20     calculate(num: number) => this;
21 }
22 // 插件实现
23 class AddPlugin implements Plugin {
24     name: 'add',
25     calculate(num) {
26         this.num = this.num + num;
27         return this;
28     }
29 }
```

```

28     }
29 }
30
31 class SubtractPlugin implements Plugin {
32     name: 'subtract',
33     calculate(num) {
34         this.num = this.num - num;
35         return this;
36     }
37 }
38
39 const myCalculator = new Calculator(5);
40 // 插件安装
41 myCalculator.use(new AddPlugin());
42 myCalculator.use(new SubtractPlugin());
43
44 myCalculator.add(5).subtract(2).result(); // 8

```

经过这个改造，未来如果要实现乘法，我们只需要新增一个插件实现即可，无需修改程序主体：

```

1 class MultiplicatiPlugin implements Plugin {
2     name: 'multiplicati',
3     calculate(num) {
4         this.num = this.num * num;
5         return this;
6     }
7 }

```

再比如我们要增加一个 help 来打印支持的计算也可以快速实现：

```

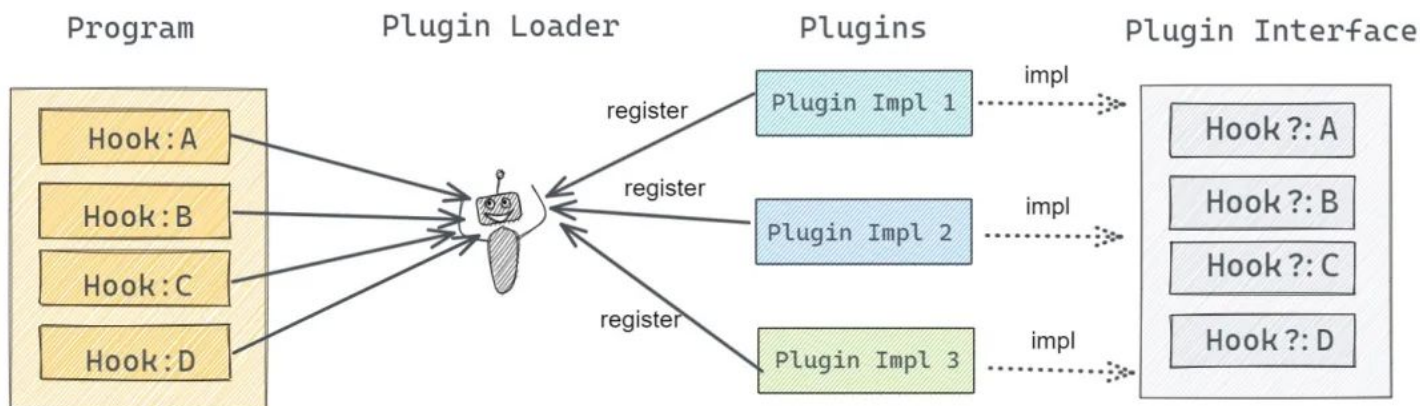
1 class Calculator {
2     ...
3     help() {
4         return `support ${this.plugins.map(plugin => plugin.name).join(',')}`;
5     }
6 }

```

通过上面的例子，从插件的角度可以分成几个部分：

- **程序主体（Program）**，我们通常也称之为插件底座，即上例中的 Calculator；
- **插件接口声明（Plugin Interface）**，即上例中的 Plugin；

- 插件实现（Plugin Implementation），即 AddPlugin, SubtractPlugin, MultiplicatiPlugin;



插件化机制用途非常广泛：

1. 表单校验，校验规则就可以是插件化的，**rules: [{required: true}, {max: 20}, {min: 10}]**
2. webpack、vue、pinia 这些框架，他的能力拓展都是通过插件化实现的

物料插件化

我们上一节提到过，物料可以分为基础数据和资源引用，也就是：

```

1  const material = {
2      type: '物料类型',
3      material: '物料资源'
4  }
  
```

我们的物料资源引用可以来自本地文件，也可以来自在线物料市场，我们暂以本地物料资源为例。

```

1  import type { BlockType } from '@types/block'
2  import QuoteBlock from '@blocks/basic/QuoteBlock.vue'
3  import HeroTitleBlock from '@blocks/basic/HeroTitleBlock.vue'
4  import ViewBlock from '@blocks/basic/ViewBlock.vue'
5  import ChartBlock from '@blocks/basic/ChartBlock.vue'
6  import ImageBlock from '@blocks/basic/ImageBlock.vue'
7  import ButtonBlock from '@blocks/external/ButtonBlock.vue'
8  import FormBlock from '@blocks/external/FormBlock.vue'
9  import NotesBlock from '@blocks/external/NotesBlock.vue'
10 import type { App } from 'vue'
11
  
```

```

12  const baseBlocks = [
13    {
14      type: 'quote',
15      material: QuoteBlock
16    },
17    {
18      type: 'heroTitle',
19      material: HeroTitleBlock
20    },
21    {
22      type: 'view',
23      material: ViewBlock
24    },
25    {
26      type: 'chart',
27      material: ChartBlock
28    },
29    {
30      type: 'image',
31      material: ImageBlock
32    }
33  ]
34  class BlockSuite {
35    private blocks = baseBlocks
36    constructor() {}
37    getBlocksMap() {
38      return Object.fromEntries(this.blocks.map((block) => [block.type, block]))
39    }
40    getBlocks() {
41      return this.blocks
42    }
43    addBlock(block: any) {
44      this.blocks.push(block)
45    }
46    hasBlock(type: BlockType) {
47      return !!this.getBlocksMap()[type]
48    }
49  }
50
51  const blockSuite = new BlockSuite()
52
53  console.log(
54    '🚀 ~ file: BlockRenderer.vue:55 ~ blockSuite.hasBlock(button):',
55    blockSuite.hasBlock('button')
56  )
57
58  blockSuite.addBlock({

```

```

59     type: 'button',
60     material: ButtonBlock
61   })
62   blockSuite.addBlock({
63     type: 'form',
64     material: FormBlock
65   })
66   blockSuite.addBlock({
67     type: 'notes',
68     material: NotesBlock
69   })
70   console.log(
71     '🚀 ~ file: BlockRenderer.vue:68 ~ blockSuite.hasBlock(button):',
72     blockSuite.hasBlock('button')
73   )
74   const blocksMap = blockSuite.getBlocksMap()
75
76   export const blocksMapSymbol = Symbol('blocksMap')
77
78   export const setup = (app: App<Element>) => {
79     const ins = {
80       install(app: App<Element>) {
81         app.provide(blocksMapSymbol, blocksMap)
82         app.config.globalProperties.$blocksMap = blocksMap
83       }
84     }
85
86     app.use(ins)
87   }
88
89   // Extensions of Vue types to be appended manually
90   // https://github.com/microsoft/rushstack/issues/2090
91   // https://github.com/microsoft/rushstack/issues/1709
92
93   // TODO: figure out why it cannot be 'vue'
94   // @ts-ignore: works on Vue 3, fails in Vue 2
95   declare module '@vue/runtime-core' {
96     export interface ComponentCustomProperties {
97       /**
98        * Access to the application's blocksMap
99        */
100       $blocksMap: string
101     }
102   }

```


编排

有了物料，我们就可以在编辑器中使用对应物料进行业务页面组装，因此我们需要封装一个物料编排编辑器，对于众多物料，我们首先就会想到需要使用**策略模式**实现不同物料的初始渲染。

```
1 <component :is="xxx" />
```

```
1 <script setup lang="ts">
2 import { toRaw } from 'vue'
3 import { Drag, Delete } from '@icon-park/vue-next'
4 import { useEnvStore } from '@stores/debug'
5 import { useAppEditorStore } from '@stores/appEditor'
6 import { smoothDnD, type DropResult, dropHandlers } from 'smooth-dnd'
7 import { arrayMove } from '@utils/array'
8
9 import { SmoothDndContainer } from '@components/SmoothDnd/SmoothDndContainer'
10 import { SmoothDndDraggable } from '@components/SmoothDnd/SmoothDndDraggable'
11 import { storeToRefs } from 'pinia'
12
13 smoothDnD.dropHandler = dropHandlers.reactDropHandler().handler
14
15 // const props = defineProps<{
16 //   type: BlockType
17 // }>()
18
19 const envStore = useEnvStore()
20 const appEditorStore = useAppEditorStore()
21
22 const { currentBlockId, blocks } = storeToRefs(appEditorStore)
23 const { selectBlock, updateBlocks } = appEditorStore
24
25 const applyDrag = <T extends any[]>(arr: T, dragResult: DropResult) => {
26   const { removedIndex, addedIndex, payload } = dragResult
27
28   const result = [...arr]
29
30   // 没做操作
31   if (addedIndex === null) return result
32
33   // 添加
34   if (addedIndex !== null && removedIndex === null) {
35     result.splice(addedIndex, 0, {
36       id: `${Math.random()}`,
37       ...payload
```

```

38     })
39 }
40
41 // 移动
42 if (addedIndex !== null && removedIndex !== null) {
43     return arrayMove(result, removedIndex, addedIndex)
44 }
45
46 return result
47 }
48 </script>
49
50 <template>
51     <smooth-dnd-container
52         drag-handle-selector=".handle"
53         group-name="blocks"
54         orientation="vertical"
55         tag="div"
56         @drop="updateBlocks(applyDrag(toRaw(blocks), $event))"
57     >
58         <smooth-dnd-draggable v-for="(block, i) in blocks" :key="block.id">
59             <div class="block-wrapper" @click.stop="selectBlock(block.id)">
60                 <!-- @vue-ignore -->
61                 <component :is="$blocksMap[block.type].material" class="block" />
62                 <div
63                     :class="[
64                         'block-wrapper-indicator',
65                         { shown: envStore.debug, selected: currentBlockId === block.id }
66                     ]"
67                 >
68                     <div class="block-toolbar" v-if="currentBlockId === block.id">
69                         <div class="block-toolbar-item handle">
70                             <drag />
71                         </div>
72                         <div class="block-toolbar-item" @click="blocks.splice(i, 1)">
73                             <delete />
74                         </div>
75                     </div>
76                 </div>
77             </div>
78         </smooth-dnd-draggable>
79     </smooth-dnd-container>
80     <!-- <div ref="containerRef">
81         <div
82             class="block-wrapper"
83             v-for="(d, i) in b"
84             :key="d.id"

```

```

85         @click.stop="appEditorStore.selectBlock(d.id)"
86     >
87         <component :is="d.material" class="block" />
88         <div
89             :class="[
90                 'block-wrapper-indicator',
91                 { shown: envStore.debug, selected: appEditorStore.currentBlockId
=== d.id }
92             ]"
93         >
94             <div class="block-toolbar" v-if="appEditorStore.currentBlockId ===
d.id">
95                 <div class="block-toolbar-item handle">
96                     <drag />
97                 </div>
98                 <div class="block-toolbar-item" @click="b.splice(i, 1)">
99                     <delete />
100                 </div>
101             </div>
102         </div>
103     </div>
104 </div> -->
105 </template>
106
107 <style scoped>
108     .block-wrapper {
109         position: relative;
110         display: flex;
111         width: 100%;
112         margin-top: 16px;
113         padding: 6px 4px;
114         border-radius: 8px;
115         background-color: var(--color-white);
116         transition: box-shadow 0.2s ease-in-out;
117     }
118
119     .block {
120         position: relative;
121         z-index: 1;
122     }
123
124     .block-wrapper-indicator {
125         content: '';
126         position: absolute;
127         width: 100%;
128         height: 100%;
129         left: 0;

```

```
130     top: 0;
131     border-radius: 8px;
132     pointer-events: none;
133     user-select: none;
134 }
135
136 .block-wrapper-indicator.shown {
137     border: 1px dashed var(--color-gray-300);
138 }
139
140 .block-wrapper-indicator.selected {
141     border: 1px solid var(--color-primary);
142 }
143
144 .block-toolbar {
145     display: flex;
146     align-items: center;
147     position: absolute;
148     left: 0;
149     top: -36px;
150     z-index: 1;
151     padding: 4px 8px;
152     gap: 4px;
153     background-color: var(--color-black);
154     border-radius: 6px;
155     pointer-events: visible;
156 }
157
158 .block-toolbar-item {
159     display: flex;
160     justify-content: center;
161     align-items: center;
162     width: 22px;
163     height: 22px;
164     border-radius: 4px;
165     color: var(--color-white);
166     cursor: pointer;
167 }
168
169 .block-toolbar-item:nth-of-type(1) {
170     cursor: grab;
171 }
172
173 .block-toolbar-item:hover {
174     background-color: var(--color-gray-800);
175     transition: all 0.2s ease-in-out;
176 }
```

```

177
178 .block-wrapper.debug:hover .block-wrapper-senior {
179     border-style: solid;
180     transition: box-shadow 0.2s ease-in-out;
181 }
182 </style>
183
184 <style lang="css">
185 .smooth-dnd-draggable-wrapper {
186     overflow: visible !important;
187 }
188 </style>
189

```

配置面板开发

配置面板用于配置页面中物料，通常我们需要重点关注的是：怎样设计是的配置尽可能规范化？

设计物料 JSON Schema

我们需要针对多样物料设计上层趋于一致的结构，这样不论是在物料编排还是出料，都可以保证数据的高度规范，有利于数据类型推导与渲染器的功能编写。

通常物料我们需要关注一下重点属性，或者说通常这个 json 我们这样设计，基本上可以满足常规需求，当然如果有更丰富的功能也可以在此基础上添加定义即可。

```

1  const materialSchema = {
2      id: '1',
3      type: 'quote',
4      label: '引述',
5      props: {
6          title: '引述文本 quote',
7          // ... 其他类型物料可以在这里有更多属性定义
8      },
9      events: {
10         onClick(ev, ctx, item) {
11             console.log(ev)
12         }
13     },
14     attrs: {},
15     // children: [] 如果你的编排选择使用树状组织，则有可能需要通过 children 知道后代元素，
16     // 当然你也可以选择平铺数据表示
17 }

```

配置栏设计

毫无疑问，配置栏也主要是基于策略模式实现，不同物料对应不同物料配置组件，但有一点需要注意，他们组件对上层的 api 要高度一致，提供相同的接近的 props 和事件。

右侧栏的本质其实就是表单，但是跟大家平常做的简单后台项目表单不同的是，右侧栏表单更复杂，通常还有动态及联动表单的需求。

```
1  <script setup lang="ts">
2  import { blocks } from '@mocks/blocks'
3  import { blocksBaseMeta } from '@constants/blocksBaseMeta'
4  import { useAppEditorStore } from '@stores/appEditor'
5  import { computed } from 'vue'
6
7  const appEditorStore = useAppEditorStore()
8
9  const blocksMap = computed(() => {
10     return blocks.reduce<Record<string, (typeof blocks)[0]>>((acc, cur) => {
11         acc[cur.id] = cur
12         return acc
13     }, {})
14 })
15
16 const currentBlockInfo = computed(() => {
17     if (!appEditorStore.currentBlockId) return null
18     return blocksMap.value[appEditorStore.currentBlockId]
19 })
20 </script>
21
22 <template>
23     <div class="app-right-panel-wrapper">
24         <template v-if="currentBlockInfo">
25             <div class="app-right-panel-header">
26                 {{ blocksBaseMeta[currentBlockInfo.type].name }}
27             </div>
28             <input :defaultValue="currentBlockInfo.props.title" />
29             <div class="app-right-panel-content">{{ currentBlockInfo.type }}</div>
30         </template>
31     </div>
32 </template>
33
34 <style scoped>
35     .app-right-panel-wrapper {
```

```

36     position: relative;
37     z-index: 4;
38     width: var(--panel-width);
39     height: 100%;
40     box-shadow: var(--color-gray-300) 1px 0 0;
41 }
42
43 .app-right-panel-header {
44     font-size: var(--font-size-normal);
45     font-weight: var(--font-weight-bolder);
46     height: 44px;
47     line-height: 44px;
48     padding: 0 16px 0 20px;
49 }
50
51 .app-right-panel-content {
52     padding: 0 16px 0 20px;
53 }
54 </style>
55

```

配置栏内层组件开发

我们上面提到，配置栏外层是一个容器，配置具体实现需要配置栏内层元素通过选中策略来确定。

`AppRightPanel` 负责数据分发与内容策略渲染，具体实现由对应物料配置器实现。

```

1  <script setup lang="ts">
2  import { blocks } from '@mocks/blocks'
3  import { blocksBaseMeta } from '@constants/blocksBaseMeta'
4  import { useAppEditorStore } from '@stores/appEditor'
5  import { computed } from 'vue'
6
7  const appEditorStore = useAppEditorStore()
8
9  const blocksMap = computed(() => {
10     return blocks.reduce<Record<string, (typeof blocks)[0]>>((acc, cur) => {
11         acc[cur.id] = cur
12         return acc
13     }, {})
14 })
15
16 const currentBlockInfo = computed(() => {
17     if (!appEditorStore.currentBlockId) return null
18     return blocksMap.value[appEditorStore.currentBlockId]
19 })

```

```

20 </script>
21
22 <template>
23   <div class="app-right-panel-wrapper">
24     <template v-if="currentBlockInfo">
25       <div class="app-right-panel-header">
26         {{ blocksBaseMeta[currentBlockInfo.type].name }}
27       </div>
28       <div class="app-right-panel-content">
29         <div>
30           {{ currentBlockInfo.type }}
31         </div>
32         <input v-if="currentBlockInfo.type === 'quote'"
:defaultValue="currentBlockInfo.label" />
33       </div>
34     </template>
35   </div>
36 </template>
37
38 <style scoped>
39 .app-right-panel-wrapper {
40   position: relative;
41   z-index: 4;
42   width: var(--panel-width);
43   height: 100%;
44   box-shadow: var(--color-gray-300) 1px 0 0;
45 }
46
47 .app-right-panel-header {
48   font-size: var(--font-size-normal);
49   font-weight: var(--font-weight-bolder);
50   height: 44px;
51   line-height: 44px;
52   padding: 0 16px 0 20px;
53 }
54
55 .app-right-panel-content {
56   padding: 0 16px 0 20px;
57 }
58 </style>
59

```



受控组件与非受控组件

输入值与状态是否双向绑定

使用 vee-validate 实现表单

vee-validate 这个库设计很不错，推荐大家使用，区别于传统表单组件，vee-validate 完全基于 composition api，他的效果跟 react-hook-form，相信用过 react 的同学都知道。😊

我们根据需求定义自定义表单内容，表单中通常包含 input、select 等表单录入组件，然后在外层封装一个大 form 容器组件，用于把收集到的数据进行管理。

我们先来看一个 vee-validate 的示例，我们需要定义一个自定义的 TextInput 组件

```
1  <template>
2    <input v-model="value" :type="type || 'text'" />
3    {{ errorMessage }}
4  </template>
5
6  <script setup>
7    import { useField } from 'vee-validate';
8
9    const props = defineProps({
10     name: String,
11     type: String,
12   });
13
14   // The `name` is returned in a function because we want to make sure it stays
15   // reactive
16   // If the name changes you want `useField` to be able to pick it up
17   const { value, errorMessage } = useField(() => props.name);
18 </script>
```

在上层使用：

```
1  <template>
2    <form @submit="onSubmit">
3      <InputText name="firstName" />
4      <InputText name="lastName" />
5      <InputText name="email" type="email" />
6      <InputText name="password" type="password" />
7      <InputText name="passwordConfirm" type="password" />
8
9      <button>Submit</button>
10    </form>
11  </template>
12
13  <script setup>
```

```

14 import { useForm } from 'vee-validate';
15 import * as yup from 'yup';
16 import InputText from './InputText.vue';
17
18 const { handleSubmit } = useForm({
19   validationSchema: yup.object({
20     firstName: yup.string().required(),
21     lastName: yup.string().required(),
22     email: yup.string().required().email(),
23     password: yup.string().required().min(6),
24     passwordConfirm: yup
25       .string()
26       .required()
27       .min(6)
28       .oneOf([yup.ref('password')]),
29   }),
30 });
31
32 const onSubmit = handleSubmit(values => {
33   alert(JSON.stringify(values, null, 2));
34 });
35 </script>

```

就很奇怪！为什么他知道下层就知道我们当前的 `form` 实例的呢？

那是借助了 `vue` 的 `provide` 和 `inject`。

xxxBlockSettingContent

针对不同 block 我们需要实现对应的编辑组件，以 `quote` 为例，我们需要配置 `quote` 的状态和内容，分别对应的表单两个字段。

```

1 const { value: status, errorMessage: statusErrorMessage } = useField(() =>
  'status');
2 const { value: content, errorMessage: contentErrorMessage } = useField(() =>
  'content');

```

通常表单是通过 `onSubmit` 触发提交，但这里我们需要修改完成后就立即将数据提交，因此我们在容器上层需要使用 `provide` 将表单变更数据的方法提供给下级元素，在各自的表单容器中，通过 `inject` 的方式将该方法注入，即可实现。

表单生成器 Form Builder 动态生成表单实现校验与提交

表单我们使用表单块实现 `FromBlock` 。

相关规则配置在右侧进行，动态表单我们借助 `vee-validate` 提供的 `useFieldArray` 实现，与 `react-hook-form` 使用基本类似。

相关实现见源码。

其他资料

<https://form.making.link/>

可视化低代码表单设计器 - FormMaking

基于 Vue 的可视化低代码表单设计器，赋能企业实现低代码开发模式；帮助开发者从传统枯燥的表单代码中解放出来，更多关注业务，快速提高效率，节省研发成本。在OA系统、考试系统、报表系统、流程管理等系统中应用广泛。

alibaba/lowcode-engine



An enterprise-class low-code technology stack with scale-out design / 一套面向扩展设计的企业级低代码技术体系

89

96

15k

3k

Contributors

Used by

Stars

Forks



<https://github.com/alibaba/lowcode-engine>

GitHub - alibaba/lowcode-engine: An enterprise-class low-code technology stack with scale-out design

An enterprise-class low-code technology stack with scale-out design / 一套面向...

sahadev/vue-component-creator-ui



拖拽式Vue组件代码生成编辑器（VCC）

2

5

2

789

216

Contributors

Issues

Discussions

Stars

Forks



<https://github.com/sahadev/vue-component-creator-ui>

GitHub - sahadev/vue-component-creator-ui: 拖拽式Vue组件代码生成编辑器（VCC）

拖拽式Vue组件代码生成编辑器（VCC） . Contribute to sahadev/vue-component-...

总结

主要介绍了**插件化**，策略模式实现组件内容，**vee-validate** 重点掌握。